```
expect(children).to match [
  { name: 'Coen',   age: a_value > 5 },
  { name: 'Daphne', age: a_value_between(3, 5) },
  { name: 'Crosby', age: a_value < 3 }
]
```

When you're matching against a string, match delegates to String#match, which accepts either a regular expression or a string:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb

```
expect('a string').to match(/str/)
expect('a string').to match('str')
```

Naturally, this matcher has an_object_matching and a_string_matching aliases.

contain_exactly

We've seen that match checks data structures more loosely than eq; contain_exactly is even looser. The difference is that match requires a specific order, whereas contain_exactly ignores ordering. For example, both of these expectations pass:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb

```
expect(children).to contain_exactly(
  { name: 'Daphne', age: a_value_between(3, 5) },
  { name: 'Crosby', age: a_value < 3 },
  { name: 'Coen',   age: a_value > 5 }
)

expect(children).to contain_exactly(
  { name: 'Crosby', age: a_value < 3 },
  { name: 'Coen',   age: a_value > 5 },
  { name: 'Daphne', age: a_value_between(3, 5) }
)
```

Like include, contain_exactly receives multiple array elements as separate arguments. It's also available as a_collection_containing_exactly.

Which Collection Matcher Should I Use?

With a half-dozen collection matchers to pick from, you may wonder which one is the best for your situation. In general, we recommend you use the loosest matcher that still specifies the behavior you care about.

Avoid Overspecification: Favor Loose Matchers



Using a loose matcher makes your specs less brittle; it prevents incidental details from causing an unexpected failure.

The flowchart on page 202 provides a quick reference for the different uses of collection and string matchers.